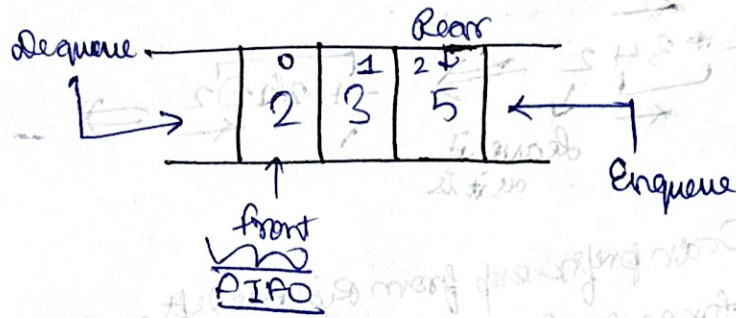


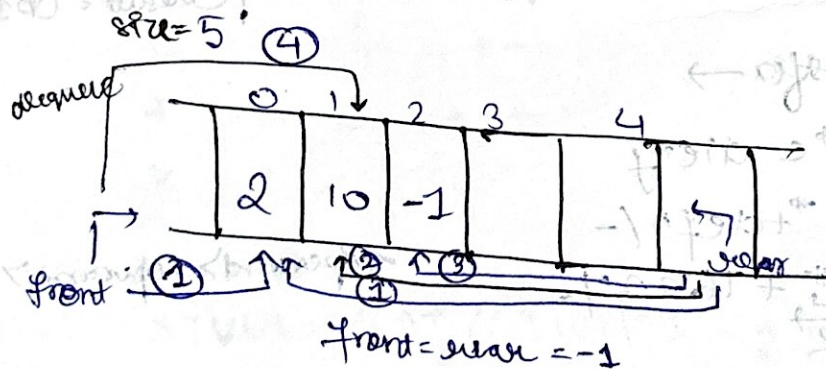
④ Queue → Linear Data Structure

Follow First In First Out (FIFO) Rule

- ① Insertion - Enqueue() ⇒ Rear / Tail (Start from one end)
- ② Deletion - Dequeue() ⇒ Head / Front (Start from another end)



- ④ Operations:
- ① Enqueue()
 - ② Dequeue()
 - ③ front() / Rear()
 - ④ isFull()
 - ⑤ isEmpty()



- ① Enqueue (2) → rear++, front++
- ② Enqueue (10) → rear++
- ③ Enqueue (-1) → rear++
- ④ Dequeue → print the value, like here 2 then front++, front = 1, rear = -1, 2 = Garbage value

⑧ Implementation using arrays →

```
#define N 5
```

```
int queue [N];
```

```
int front = -1;
```

```
int rear = -1;
```

```
void enqueue (int x) {
```

```
    if (rear == N-1) {
```

```
        printf("Overflow");
```

```
    } else if (front == -1 && rear == -1) {
```

```
        front = rear = 0;
```

```
        queue [rear] = x;
```

```
    } else { rear++;
```

```
        queue [rear] = x;
```

```
}
```

```
void dequeue () {
```

```
    if (front == -1 && rear == -1) {
```

```
        printf("Empty Queue, Underflow");
```

```
    } else if (front == rear) {
```

```
        front = rear = -1;
```

```
    } else { printf("dequeued element = %.d", queue [front]);
```

```
        front++;
```

```
}
```

```
}
```

```
void display () { int i;
```

```
    if (front == -1 && rear == -1) {
```

```
        printf("Underflow");
```

```
    } else {
```

```
        for (i = front; i < rear+1; i++) {
```

```
            printf("%.d", queue [i]);
```

```
        }
```

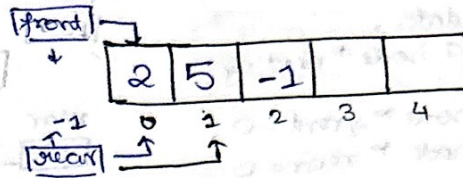
```
}
```

```
void peek () { if (condn
```

```
    else {
```

```
        printf("%.d", queue [front]);
```

```
}
```



[Insert 2]

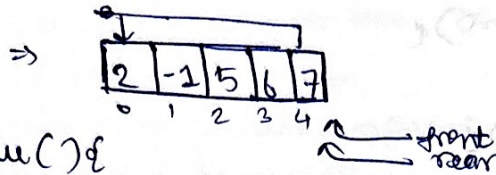
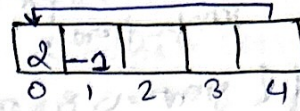
After Inserting 2, we want to insert 5 so first rear++ then insert. Same for inserting -1.

[Agar front = rear = 0, ek element sirf use delete krna to -1 ko change krna]

else if $(\text{rear} + 1) \% N == \text{rear}$;

{
 $\text{queue}[\text{rear}] = x$
}

$(0+1) \% 5 \rightarrow 1 = \text{rear}$
 $\text{queue}[1] = x$

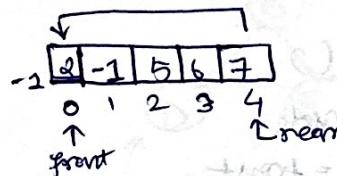


void dequeue() {

if $(\text{front} == -1 \ \& \ \text{rear} == -1)$ {
 printf("Queue is empty");

else if $(\text{front} == \text{rear})$ {
 $\text{front} = \text{rear} = -1$;

else {
 $\text{front} = (\text{front} + 1) \% N$;



void display() {

1) $\text{int } i = \text{front}$;

2) while $(i \neq \text{rear})$ {
 printf("%d ", $\text{queue}[i]$);

$i = (i + 1) \% N$;
 printf("%d ", $\text{queue}[\text{rear}]$);

Linked list :- struct node {
 int data;
 struct node * next;
};
struct node * front = 0;
struct node * rear = 0;

void enqueue (int x) {

 struct node * newnode;
 malloc & all
 newnode -> data = x;
 newnode -> next = 0;

 if $(\text{rear} == 0)$ {
 $\text{front} = \text{rear} = \text{newnode}$;
 $\text{rear} \rightarrow \text{next} = \text{front}$;

 } else {
 $\text{rear} \rightarrow \text{next} = \text{newnode}$;
 $\text{rear} = \text{newnode}$;
 $\text{rear} \rightarrow \text{next} = \text{front}$;

void dequeue() {

struct node * temp;
temp = front;

if (front == 0 & rear == 0) {

Empty

else if (front == rear) {

front = rear = 0;

free(temp); }

else {
front = front -> next;
rear -> next = front;
free(temp); }

void display() {

① temp = node

② temp = front

③ else → temp =

while (temp -> next != front) {

printf("%d", temp -> data);

temp = temp -> next;

printf("%d", temp -> data); }

★ dequeue →

front & Rear

Delete and Insert from both ends.

⑥ Types of dequeue ⇒

① Input Restricted

→ Insertion from one end, deletion from both.

② Output Restricted

→ deletion from " " , Insertion " " .

⑦ Applications ⇒

① Redo & Undo

② Multiprocessor scheduling.